

P R O M P T . C O N T E X T . C L A U D E C O D E

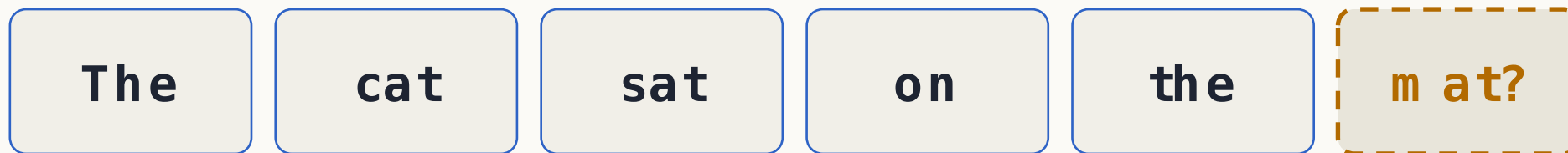
Engineering Predictable AI

From "type and hope" to a real workflow you can ship with.

2-hour lesson • Week 16 • Full-Stack • Data Analytics • GenAI

What is an LLM, really?

One-line answer: a pattern-matching machine trained on the internet. Predicts the next token, one at a time.



predict

WHAT IT IS

- A pattern-matching machine trained on internet text
- Predicts the next token, one at a time
- Do that 500 times → get a paragraph
- Very good at what looks likely, given what came before

WHAT IT ISN'T

- A database — does not "know" facts, only patterns
- A search engine — cannot look things up unless you give it a tool
- An oracle — will guess plausibly rather than say "I don't know"
- Conscious — no memory between chats, no "self"

THE PUNCHLINE

If you don't tell it, it doesn't know. Every fact you need must be in the context — or the model will guess.

What is a prompt?

Everything you send to a language model in ONE call. Not just your question — the whole envelope.

The chat box you type in

↓ what the API actually receives ↓

SYSTEM

Setup rules. Persona. Output format. What it must never do.

USER

Your question THIS turn. The input, the task.

ASSISTANT

Model's reply — AND every prior reply (the history).

SYSTEM : You are a sentiment extractor.
Return ONLY JSON : { "sentiment": "pos|neg|neu" }

USER : "Loved the coffee, hated the wait."

ASSISTANT: { "sentiment": "neg" }

Three roles. One model call. One reply.

That whole block IS the prompt.

THE CATCH

A chat box shows you only the USER half. The system message is hidden — set by the app vendor. The whole point of this class: learn to control all three.

IF YOU REMEMBER ONE THING

A prompt is a three-part envelope: System + User + Assistant. The API sees all three. You only see one of them in a chat box.

The strict anatomy of an API call

S**SYSTEM**

TOP-PRIORITY RULES

Persona, output format, hard constraints. You write it ONCE — it sits in front of every turn. Highest authority.

U**USER**

THE INPUT

What the human (or your app) sends THIS turn. The actual question, the row to process, the file to summarize.

A**ASSISTANT**

THE EXECUTION

Model's reply. Also includes every prior reply in the conversation — that's the model's memory of this session.

VOCAB CHEAT

Anthropic "system" = OpenAI "developer" = Google "system_instruction". Different name on the parameter. Exact same role.

What you CAN and CANNOT control

Every debugging session starts here. Know which lever you're pulling before you pull it.

YOU CONTROL

- System prompt (rules, persona)
- User message (the ask)
- Model choice (Claude/GPT/Gemini)
- Temperature (randomness)
- max_tokens (output cap)
- Tools + their schemas
- Structured output schema
- Few-shot examples

YOU INFLUENCE

- Format compliance (JSON, bullets)
- Refusal behavior
- Tone and register
- Which sources it cites
- How much reasoning it shows

YOU DON'T CONTROL

- Exact word choice
- Internal reasoning steps
- Whether it used your context vs training
- Latency guarantees
- Cost precision
- The training cutoff date

THE DEBUGGING RULE

If a prompt fails, ask: was it a CONTROL knob I forgot to turn, or an INFLUENCE lever I can't force? The answer changes what you do next.

Same three roles • three different surfaces

Where you actually meet System / User / Assistant — and which one each tool hides.

CHAT.AI

claude.ai • chatgpt.com

GOOD FOR

- ▶ Easy to start
- ▶ Best for one-off questions

WATCH OUT

- ✗ System message HIDDEN
- ✗ No JSON mode you can call
- ✗ No way to script it

PLAYGROUND / WORKBENCH

console.anthropic.com • aistudio.google.com

GOOD FOR

- ▶ All 3 roles visible
- ▶ Temperature / JSON mode visible
- ▶ You can iterate prompts

WATCH OUT

- ✗ No tools / no file system
- ✗ No persistent state
- ✗ Not multi-turn-friendly

CLAUDE CODE

CLI • plan mode • MCP • subagents

GOOD FOR

- ▶ Reads files
- ▶ Edits files
- ▶ Runs commands
- ▶ Plan mode
- ▶ CLAUDE.md memory

WATCH OUT

- ✗ CLI feel takes a day
- ✗ Costs more (uses tools)

TODAY YOU SEE ALL THREE

Demo: same prompt in Claude.ai chat, in the Workbench, and in Claude Code. Same model. Three different surfaces.

The 6-ingredient prompt

Memorize these. If you can't fill in at least 5 of the 6 — your prompt isn't done.

01**ROLE**

Who is the model pretending to be?

"You are a senior security engineer."

02**CONTEXT**

What can't it guess? Audience, domain, file.

"My audience is busy CFOs."

03**TASK**

ONE verb. If your task has "and" twice → 2 prompts.

"Classify each issue as bug | style | perf."

04**CONSTRAINTS**

Length, tone, what to AVOID.

"≤ 80 words. No marketing language. Never invent stats."

05**OUTPUT FORMAT**

Exact shape. SHOW a schema.

{ "label": ..., "confidence": ... }

06**EXAMPLES**

0, 1, or a few exemplars.

Input: ... → Output: ...

THE AUDIT

Look at any prompt. Tick which of the 6 it covers. Anything below 5 = rewrite.

#1 • Vague vs ingredient-complete • 10 min

PROMPT A — the bad one

tell me about climate

4 ingredients missing.

Long, generic, no audience.

PROMPT B — engineered

You are a science journalist writing
for busy executives.

Write 3 bullets on grid-scale battery
storage in 2026.

Each bullet ≤ 25 words.

Plain English. No marketing language.

End with one line: "Bottom line: ..."

STEPS

1. Paste PROMPT A in any playground. Run. Read the first two sentences.
2. Paste PROMPT B in a NEW chat. Run.
3. Side-by-side: which would you send to your boss?
4. Bonus: same two prompts in a SECOND playground — same outcome?

WHAT TO NOTICE

Same model. Same 30 seconds of typing. The 6 ingredients turn unusable output into shippable output.

Prompt engineering • what & why

*Designing the prompt so the model **RELIABLY** produces useful output.*

Three words matter: DESIGNING (not hoping). RELIABLY (not lucky). USEFUL (not impressive).

10× SAME COST

Same model. Same dollar. Same latency. A vague prompt and an engineered one return wildly different output. The prompt is the lever.

REPEATABLE

Anyone can stumble onto one good answer. Engineering means the same prompt works on Tuesday AND Friday, dev AND prod.

SHIPPABLE

A prompt your code can rely on is a prompt you put in front of customers. Without it, AI features stay demos forever.

TODAY'S DEAL

Two hours. You leave with the 6-ingredient audit checklist + a workflow you can use in Claude Code from day one.

The right altitude for a prompt

There is a Goldilocks zone for how much you spell out. Miss it in either direction and the model breaks.

TOO VAGUE

"Be helpful and use the appropriate tools."

Model has nothing concrete to latch onto. Improvises. You get a coin toss every run.

RIGHT ALTITUDE

*"Classify each message as SALES / SUPPORT / SPAM .
Return JSON . Reasoning in one sentence .
Ask clarifying questions if the message is empty ."*

You define outcomes and shape. Model fills in the how. Works across edge cases you never listed.

TOO PRESCRIPTIVE

"If billing AND refund AND amount > \$100 AND user is EU AND day is Mon → call escalate_tool()"

Model can't deviate from your bad if/then tree. Every unlisted case breaks. You are debugging spaghetti in prose.

THE RULE

Define outcomes and shape. Let the model reason about the "how". Your job is the goal, not the algorithm.

Force JSON • three providers, three syntaxes

Same idea everywhere. Different parameter names. Pick the native mode when it exists.

Anthropic • Claude

tool_use with input_schema
(or structured_outputs
in newer SDKs)

+ PREFILL TRICK:
start assistant turn with `{`
(only Claude does this)

OpenAI

```
response_format: {
  type: "json_schema",
  json_schema: { ... },
  strict: true
}
```

Google • Gemini

```
response_mime_type:
  "application/json"

response_schema: { ... }
```

UNIVERSAL FALLBACK

Paste schema in system message + include ONE filled-in example + end user turn with: "Return ONLY valid JSON. No prose, no fences."

IF YOUR CODE PARSES IT

Use native JSON mode. Always include a filled-in example. The model copies your example more carefully than your words.

Grounding • remove the chance to guess

The fix for hallucinations is almost never "tell the model not to lie." It's "remove the situation where it has to guess."

01

PROVIDE

Paste the source document into the prompt.

02

SCOPE

"Answer ONLY using the text below."

03

ESCAPE

"If not in source, reply: NOT_IN_SOURCE."

04

EVIDENCE

"Quote the sentence that supports your claim."

SYSTEM :

Answer ONLY using < source> .

Quote the sentence that supports each claim .

If not in < source> , reply EXACTLY:
NOT_IN_SOURCE

USER:

< source>

Germany aims for 80% renewable electricity by 2030...

< /source>

Question: What share by 2030?

THE ESCAPE HATCH

Without "NOT_IN_SOURCE" the model has no legal way to say "I don't know" — so it invents.

#2 • Get strict JSON out • 12 min

Schema your model must hit:

```
{
  "sentiment": "pos" | "neg" | "neu",
  "topics": [string],
  "confidence": number // 0.0 - 1.0
}
```

Example:

Input: "Loved the coffee,
hated the wait."

Output: { "sentiment": "neg",
"topics": ["coffee", "service"],
"confidence": 0.78 }

STEPS

1. Pick any short text — tweet, review, email.
2. Get the model to return ONLY the JSON above.
3. Use native mode (response_format / response_schema / tool_use) OR Claude prefill `{`.
4. Paste output into jsonlint.com — must validate.
5. Stretch: same prompt → same JSON in a 2nd provider.

IF IT FAILS

Output wrapped in ``json fences? Add "No fences" to prompt.
Confidence always 0.95? Add "Use real confidence, do not default."

THE BAR

If `json.loads()` throws, your prompt is not done. Don't fix the JSON in code — fix the prompt.

You came for prompt engineering.

You'll leave calling it context engineering — and knowing why.

"Prompt engineering was always a little bit ridiculous.

Context is king."

— Tim Berglund · Confluent · Feb 2026

EVERYONE IS SAYING IT

Anthropic

Context engineering is the natural progression of prompt engineering.

Google

Focus on context, not clever prompts. Same info a new hire would need.

LangChain

The model is the CPU. The context window is RAM. Manage it like memory.

Confluent

Prompt is one piece. Context is the whole system.

WHAT YOU LEAVE WITH

A six-ingredient prompt · a JSON-output bot in a playground of your choice · and a mental model where prompt is one lever inside a context system you now know how to engineer.

From prompts • to context windows

Why the field is moving — and what stays the same.

2022 - 2023

CHAT ERA

The unit was a PROMPT

You typed → the model replied → one turn. The prompt WAS the conversation. Skill = writing the right one prompt.

2024 - 2026

AGENT ERA

The unit is a CONTEXT WINDOW

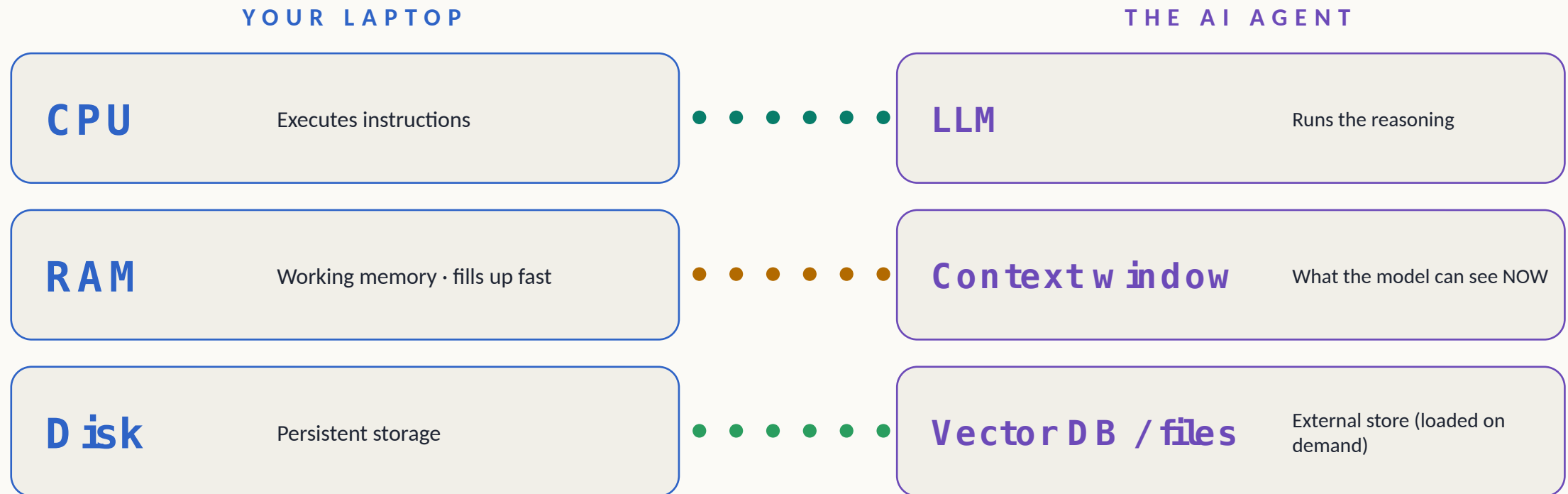
Models call tools, read retrieved docs, hold long conversations. SIX things compete for space in every call — the prompt is just one.

WHERE THIS LESSON FITS

Prompt engineering is still the foundation. Context engineering is built on top. We do 5 min of context preview now → deep dive: Week 10 + the MCP day.

The LLM is an OS • the context window is RAM

LangChain: the model is the CPU • the context window is RAM • the vector DB is disk.



THE MENTAL MODEL

When RAM fills up, your laptop lags. When the context window fills up, the model forgets. Everything we do next is memory management.

Anatomy of a context window · 6 competing components

Six things fight for space every turn. Half you control, half grows on you.

STATIC ZONE (you control) — kept up top for prefix caching

DYNAMIC ZONE (grows every turn) — the noisy tail



← RAM allocation · every token in every call →

WATCH OUT · By turn 10 of an agent run, the dynamic tail can suffocate the system prompt entirely.

THE ORDERING RULE

Static content up top (prefix cache = 10x cheaper). Dynamic content at the bottom (easier to trim). Order matters as much as content.

Four strategies • Write • Select • Compress • Isolate

Every context-management technique in the industry fits into one of these four buckets. Framework: LangChain.

WRITE	Agents forget.	Persist outside the window: scratchpads • rules files (claude.md) • memory extraction.
SELECT	Not everything is relevant now.	Load what THIS step needs: agentic RAG • RAG over tool defs • episodic/semantic/procedural memory.
COMPRESS	Context grows every turn.	Shrink before, during, after: chunking • running summaries • drop old tool outputs.
ISOLATE	Contexts contaminate each other.	Give focused subtasks their own window: sub-agents • phase resets • separate planner/coder sessions.
THE MENTAL SHORTCUT		
When your agent misbehaves, ask which of these four you skipped. The answer will name itself.		

Four ways context goes wrong

Named failure modes make debugging fast. When your agent misbehaves, one of these is the diagnosis. Framework: Drew Bruyne, 2025.

POISONING

a hallucination or tool error propagates through every next step

FIX • PRUNE • validate tool outputs • drop failed attempts

DISTRACTION

the model over-fits to recent history • stops using training

FIX • COMPRESS • summarize aggressively • reset when possible

CONFUSION

too many tools • model picks the wrong one even if it fits

FIX • SELECT • dynamic tool set • RAG over tool descriptions

CLASH

new info contradicts what is already in context

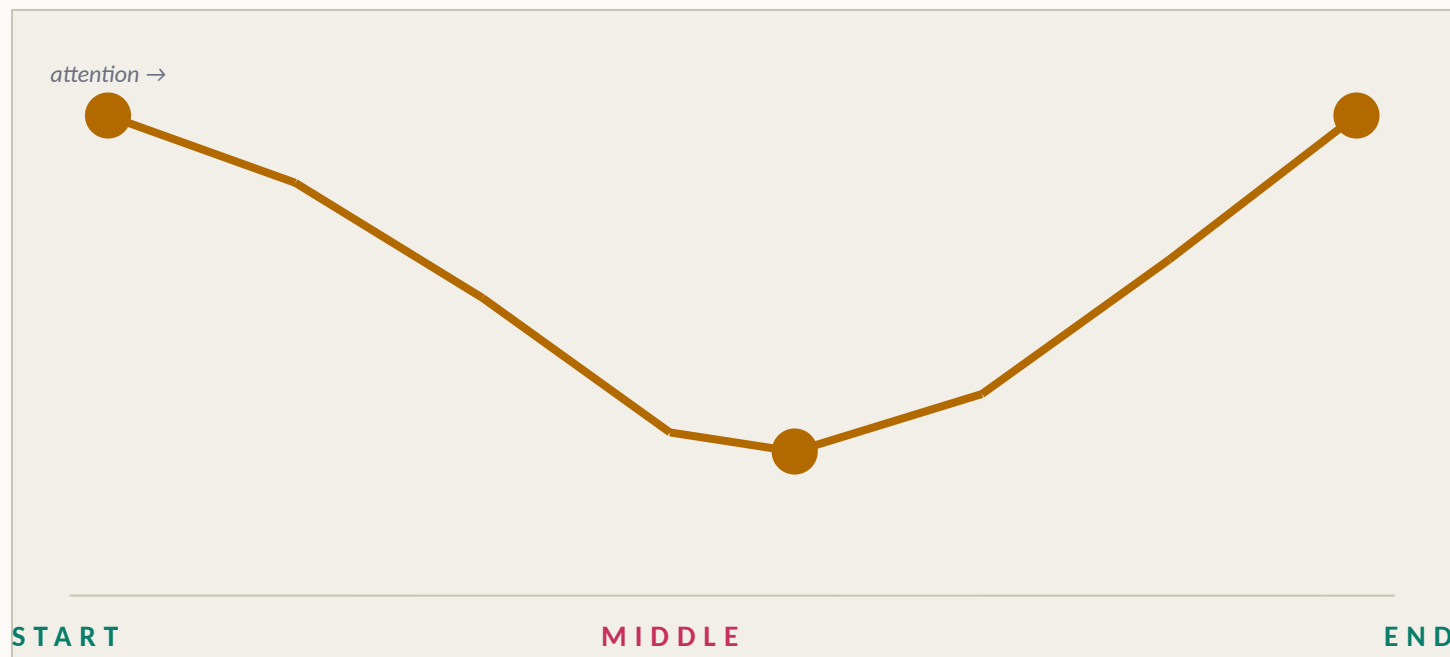
FIX • AUTHORITY ORDER • system > retrieved > history

THE DIAGNOSTIC HABIT

Bad agent behavior isn't random. Name the failure mode first • pick the strategy from slide 17 • fix it.

Lost in the middle • the U-curve • the receipts

Models weight tokens at the *START* and the *END* of the context heavier than the middle. This is not a bug you can turn off — it emerges from how attention works.



THE RECEIPTS

Chroma, 2025

18 frontier models tested. Every one degrades well below the stated context window limit.

Liu et al., NAACL

30+ percentage-point accuracy drop when the same fact moves from position 1 to the middle.

The rule of thumb

Aim for 60–70% of the window. Fuller than that = middle gets ignored.

THE 60-70% RULE

Bigger context is not always better. Stop treating "million-token window" as a solution — treat it as a warning.

#3 • Feel the U-curve • 12 min

The U-curve isn't theoretical. In 12 minutes you'll measure it yourself.

1 SETUP

Open the shared doc "hands-on-3-haystack.txt" — a 3,000-word article. Note the sentence at the very start.

2 RUN

Paste the article into 3 SEPARATE chats. In chat A, ask about the START sentence. In chat B, about a MIDDLE sentence (marked in the doc). In chat C, about the END sentence.

3 COMPARE

Score each answer 0-2 (0 = wrong, 1 = partial, 2 = exact). Report your MIDDLE score in chat. We'll compute the class average live.

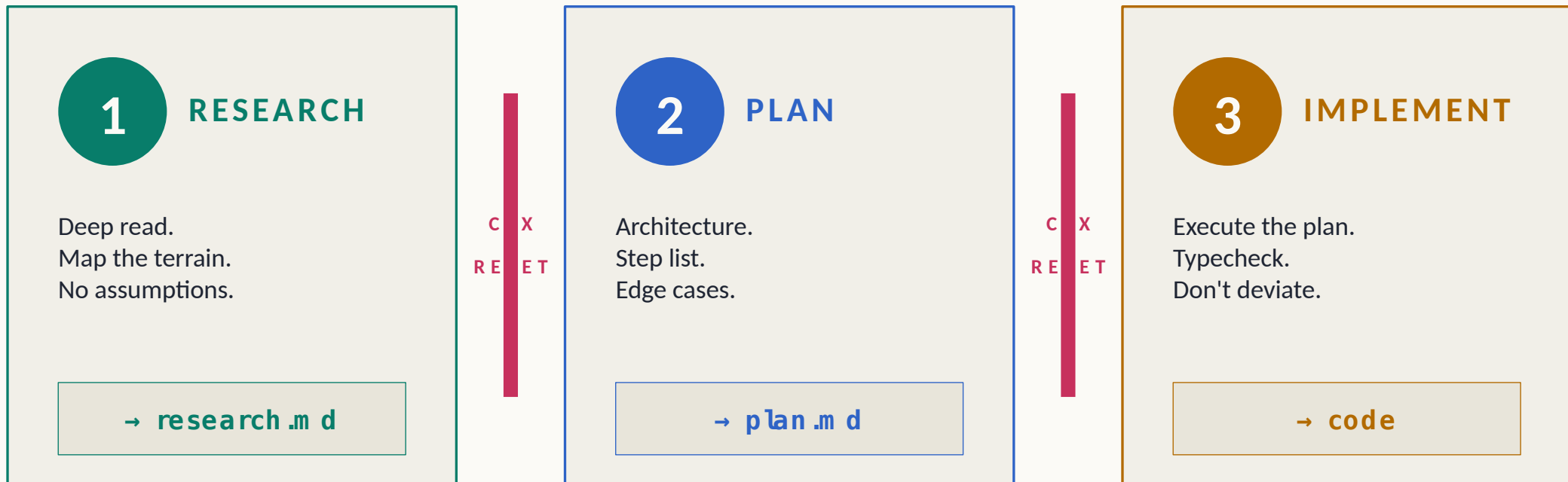
BEFORE YOU RUN • Predict on paper: which position (start / middle / end) will score lowest? Bet a coffee.

WHAT WE MEASURE

You'll see the U-curve appear as a scatter of the room's scores. Middle loses. Every time. Not a study — your data.

The 3-phase loop • with context reset barriers

The most reliable pattern for any long agent task: split it into three phases separated by context resets. Human Layer shipped 35k lines of Rust in one 7-hour session using this.



THE CORE INSIGHT

Each phase produces a compacted artifact. Each new phase opens a fresh window containing ONLY that artifact. Zero contamination.

10 tips to pin to your wall

The whole class distilled into ten rules-of-thumb. Print. Pin. Reread every Monday.

1 If your prompt has 3+ IF/THENs, rewrite as a goal.

2 temperature = 0 for anything code parses. 0.7 for humans.

3 Always set max_tokens. Never leave it default.

4 Vague adjectives are bugs. Replace with specific person or number.

5 Every prompt needs an escape hatch (e.g., "reply NOT_IN_SOURCE").

6 Static content up top. Prefix caching is 10x cheaper.

7 Aim for 60-70% context utilization. Fuller = middle ignored.

8 Name the failure mode before you fix it (Poison/Distract/Confuse/Clash).

9 Reset context between phases. Fresh window per artifact.

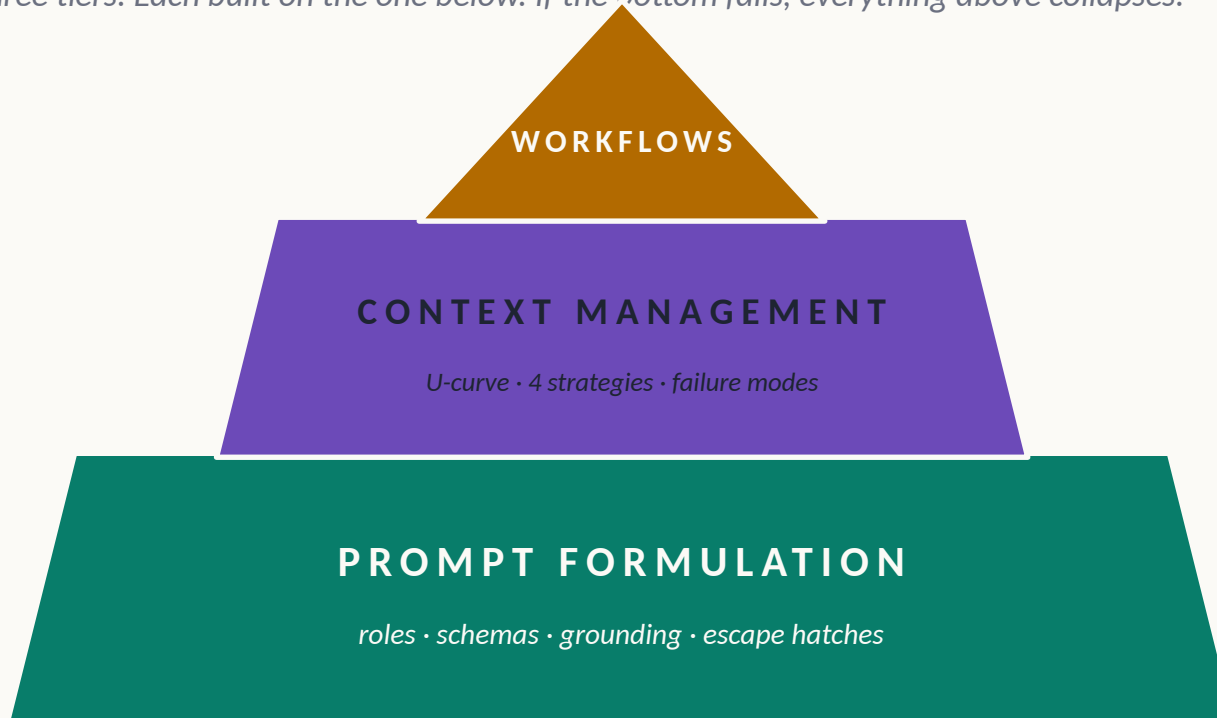
10 Prompt is the atom. Master it before you scale.

THE META-TIP

If you remember one thing: name the primitive (prompt / context / workflow) before you touch it. Debugging speed $\times 5$.

The hierarchy of AI engineering

Three tiers. Each built on the one below. If the bottom fails, everything above collapses.



IF THE BOTTOM FAILS

Every tier above it fails too. A pyramid without a base is just gravity.

THE RECIPE

A great autonomous session = great prompt formulation · scaled by context management · wrapped in workflow architecture.

WHERE TO INVEST

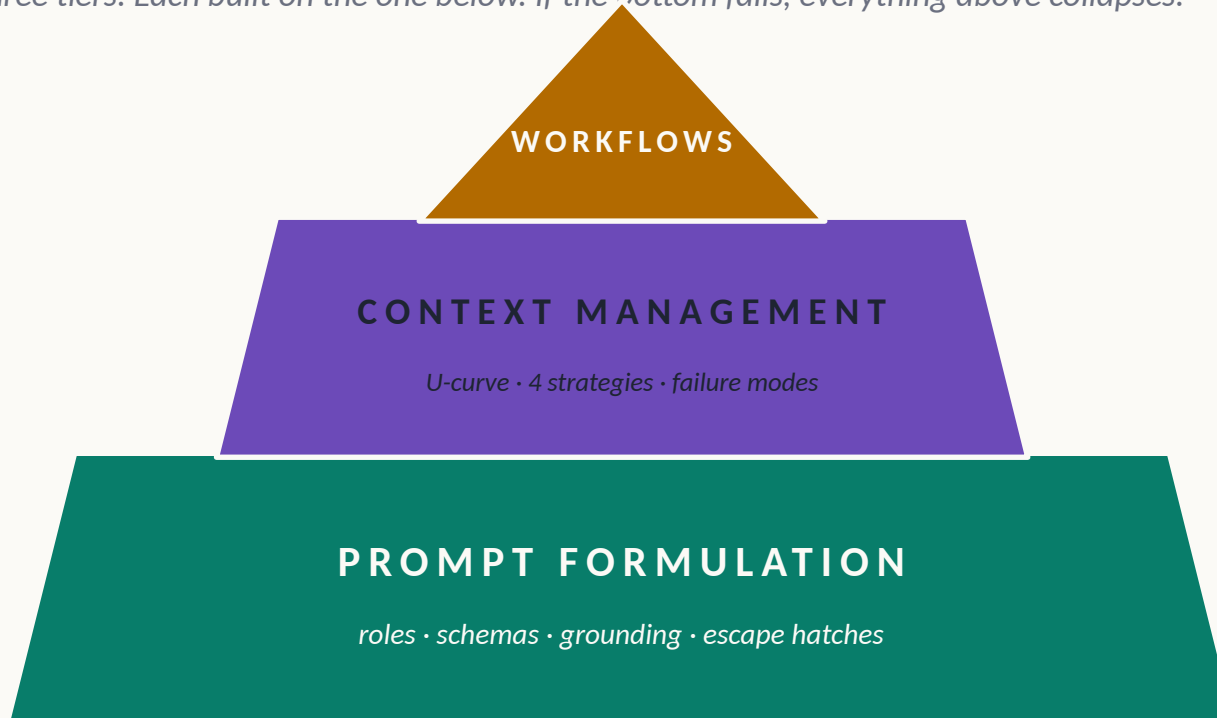
Beginners: master the bottom tier. Bootcamp week 20+: middle tier. Post-graduation: workflows.

THE FRAME

Prompt is the atom. Context is the molecule. Workflow is the organism. You are here to master the atom first.

The hierarchy of AI engineering

Three tiers. Each built on the one below. If the bottom fails, everything above collapses.



IF THE BOTTOM FAILS

Every tier above it fails too. A pyramid without a base is just gravity.

THE RECIPE

A great autonomous session = great prompt formulation · scaled by context management · wrapped in workflow architecture.

WHERE TO INVEST

Beginners: master the bottom tier. Bootcamp week 20+: middle tier. Post-graduation: workflows.

THE FRAME

Prompt is the atom. Context is the molecule. Workflow is the organism. You are here to master the atom first.